

交易系统时间配置设计

版本：v2 重构稿 范围：仅覆盖"时间配置"。杠杆、网关路由独立成文。品种与分类的完整建模由品种模块独立维护，本文档仅约定接入契约。
时间基线：全库统一 **UTC**，无时区属性。前端展示再做时区转换。

一、设计目标与原则

原则	含义
配置即所见即所得	运营所写即所得，不引入隐式行为；任何最终时段都可在 <code>t_daily_session</code> 中直接看到
运行时零计算	交易引擎只查快照表 + 覆盖表，不做规则求值、路径解析、时区转换
全 UTC	配置层、计算层、运行层统一 UTC 绝对分钟数；时区仅出现在展示层
DST 半自动	引入 <code>dst_profile</code> + <code>t_dst_schedule</code> ，运营无需每年为每个品种手工新增两个版本
审计与合规	配置变更、覆盖操作全审计，字段覆盖 MiFID II / SEC 17a-4 要求
可观测	预计算运行、覆盖生效、缓存刷新均有运行日志与监控指标

二、整体架构

2.1 三层架构

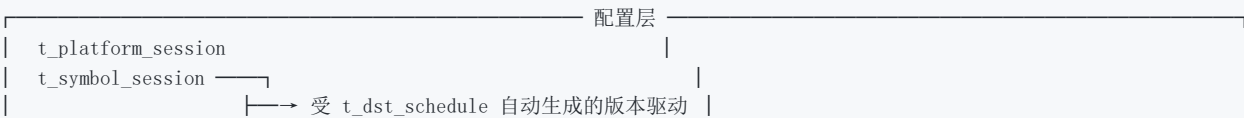
层级	职责	关键表
配置层	定义平台、品种、特殊日期的静态规则	<code>t_platform_session</code> / <code>t_symbol_session</code> / <code>t_special_session_rule</code> / <code>t_special_session</code> / <code>t_exchange_holiday</code> / <code>t_dst_schedule</code>
计算层	每日预计算，合并配置层 + 三阶段管道，生成绝对 UTC 时段快照	<code>t_daily_session</code> / <code>t_snapshot_run_log</code>
运行层	交易引擎仅查询快照表与实时覆盖表，微秒级响应	<code>t_daily_session</code> / <code>t_session_override</code>

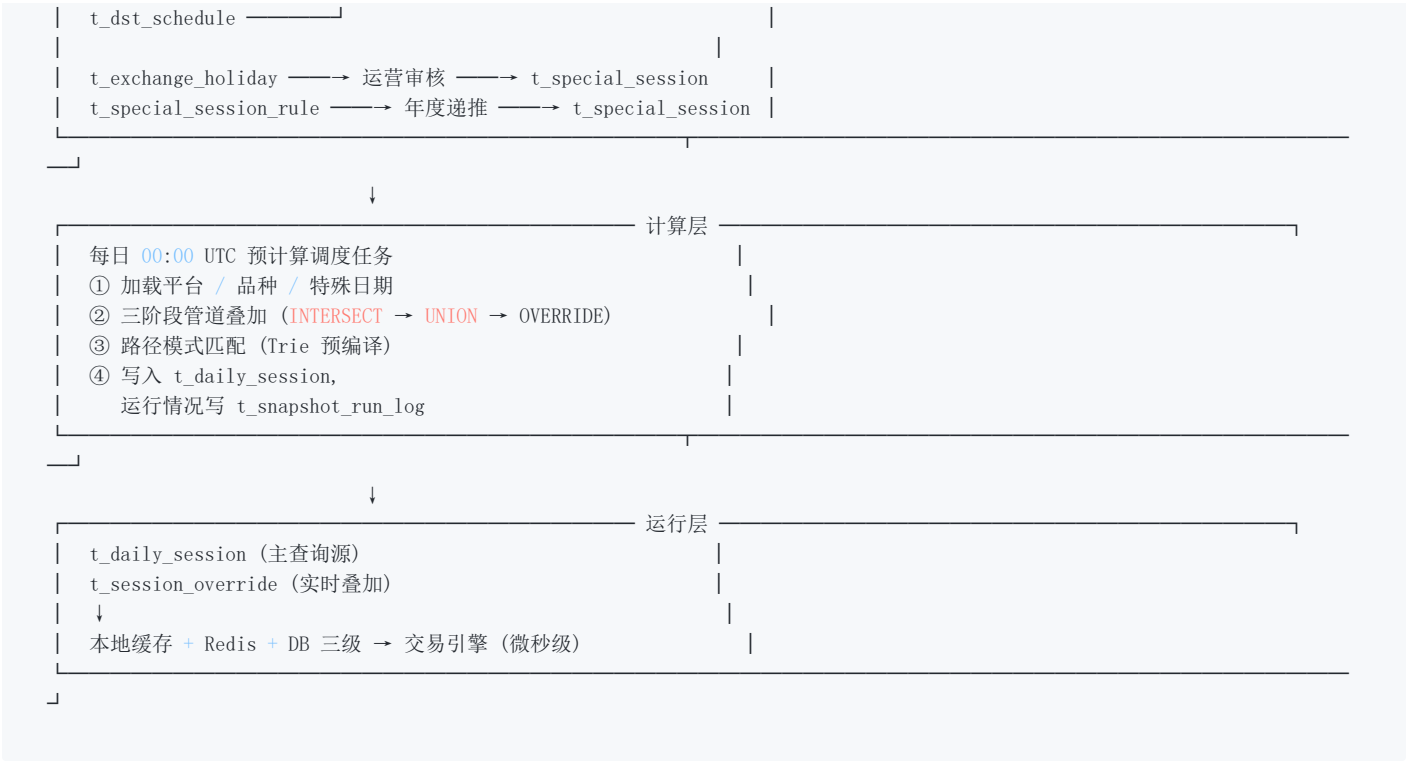
2.2 时间约束关系

平台时段 $\cap$ 品种时段 $\cap$ 特殊日期调整 $\rightarrow$ `t_daily_session` $\rightarrow$ 实时覆盖叠加 = 实际可交易状态

外层约束内层：品种时段不得超出平台时段范围；特殊日期不能突破平台时段（除非显式 **OVERRIDE**）；实时覆盖优先级最高，可强制改写任何静态结果。

2.3 数据流概览





三、核心约定（全局共识）

3.1 时间统一 UTC

所有时间字段使用 UTC 存储，不带时区属性。DATETIME(3) 精度到毫秒。

3.2 时间偏移：基于 UTC 00:00 的绝对分钟数

时刻	分钟数	示例
00:00	0	当日起点
09:30	570	美股开盘 (UTC)
16:00	960	美股收盘 (UTC)
22:00	1320	跨日前段起点
23:59	1439	当日最后一分钟
24:00 (次日 00:00)	1440	次日起点
29:00 (次日 05:00)	1740	跨日时段次日端点示例
48:00 (次日 24:00)	2880	跨日上限

3.3 跨日时段：单行 $close_min \in [0, 2880]$

允许 $close_min > 1440$ 表示跨日次日凌晨。如周日 22:00 → 周一 05:00 在 `t_symbol_session` 中表达为一行：`week_day=7, open_min=1320, close_min=1740`。

选择动机：与 MT5 Symbol Sessions、IB Trading Hours 业界主流一致；查询某时刻是否在时段内时由查询层统一做日期归一化（`if minute_of_week < open_min: minute_of_week += 10080`），无需 UNION 两行。

约束：`open_min ∈ [0, 1440]`、`close_min ∈ [open_min+1, 2880]`，且 `close_min - open_min ≤ 1440`（不允许超过 24 小时单段）。

3.4 session_type 静态时段 vs 运行时状态分离

为对齐 FIX TradingSessionStatus(340) 中 "Trading Session" 与 "Trading Session SubID" 的分层语义，本设计严格区分两类枚举：

静态时段类型（出现在 `t_platform_session` / `t_symbol_session` / `t_special_session*` / `t_daily_session.session_type`）：

静态类型	含义	FIX SessionSubID 对应
PRE_OPEN	盘前：可挂单不可成交	1 (Pre-Trading)
OPEN	正常连续交易	3 (Continuous)
PRE_CLOSE	收盘前提醒区	— (子状态)
CLOSE_ONLY	仅平仓	6 (Closing Auction) 近似
AUCTION	集合竞价	2 (Opening Auction) / 4 (Closing Auction)
POST_CLOSE	盘后	5 (Post-Trading)

运行时状态类型（仅出现在 `t_session_override.override_session_type` 与 `t_daily_session.runtime_state`）：

运行时类型	含义	FIX TradSesStatus(340) 对应
HALT	交易暂停（交易所/风控/监管发起）	1 (Halted)
MAINTENANCE	系统维护	1 (Halted) + Reason=Maintenance
QUOTE_ONLY	仅行情无交易	— (扩展)

静态类型描述"计划中的时段类型"；运行时类型描述"当下被覆盖到的实际状态"。两者语义不互相污染，运营配置时只接触静态类型，覆盖操作只接触运行时类型。

严重程度排序（运行时级联评估使用）：`HALT > CLOSE_ONLY > MAINTENANCE > QUOTE_ONLY > PRE_OPEN > AUCTION > POST_CLOSE > OPEN`

3.5 时区元数据契约（依赖品种模块）

本文档不存储品种的时区信息，假定品种模块提供以下字段：

字段	类型	含义
<code>primary_exchange</code>	VARCHAR(16)	主交易所 MIC 码（ISO 10383），如 <code>XYNS</code>
<code>primary_timezone</code>	VARCHAR(64)	IANA TZ ID，如 <code>America/New_York</code>
<code>dst_profile</code>	VARCHAR(16)	DST 归属，详见第十一节

时间设置的预计算任务读取这三个字段，用于：

- 1. 运营录入交易所公告时的本地时间 → UTC 转换辅助
- 2. DST 自动版本生成
- 3. 前端按客户时区展示

3.6 数据库通用约定

项	约定
引擎	InnoDB
字符集	utf8mb4 / utf8mb4_unicode_ci

项	约定
主键	id BIGINT UNSIGNED AUTO_INCREMENT
时间戳	created_at / updated_at 默认 CURRENT_TIMESTAMP(3)
状态字段	status TINYINT UNSIGNED 默认 1（1=启用 0=禁用）
命名	表名 t_ 前缀，索引 idx_ 前缀，唯一键 uk_ 前缀

后续各表 DDL 不再重复以上通用部分。

四、品种模块接入契约

品种与分类的完整建模由品种模块独立维护。本节仅描述时间设置对品种模块的依赖与契约。

4.1 依赖接入点

本文档对品种模块只有两处依赖：

接入点	类型	用途
symbol_id	BIGINT UNSIGNED	品种主键，关联 t_symbol_session / t_special_session / t_daily_session
path_pattern	VARCHAR(256)	品种分类路径，用于特殊日期 / 覆盖的批量匹配

4.2 path_pattern 匹配语法

glob 风格，预计算阶段编译为 Trie，运行时零路径解析开销：

模式	匹配	示例
*	单层任意	Stocks/US/* 匹配 Stocks/US/NYSE 不匹配 Stocks/US/Tech/AAPL.US
**	多层任意	Stocks/US/** 匹配上述两种
字面量	精确匹配	Stocks/US/Tech/AAPL.US 仅匹配该叶子

匹配冲突时按"最长路径优先 > priority 高优先 > created_at 后优先"决胜。

4.3 路径规范参考

```
/
├── Forex/{Major, Minor, Exotic}/
├── Metals/
├── Energy/
├── Crypto/
├── Stocks/{US, UK, JP, CN} / {NYSE, NASDAQ, Tech, ...} /
├── Indices/
├── Futures/
└── Options/
```

第一层资产大类，第二层市场 / 交易所，第三层板块 / 细分，叶子节点为具体品种。后续若需要对齐 ISO 10962 CFI 第一字符，由品种模块完成。

五、平台交易时段

5.1 表结构

SQL

```
CREATE TABLE `t_platform_session` (
  `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  `week_day` TINYINT UNSIGNED NOT NULL
    COMMENT '星期几(1=周一 7=周日)',
  `seq_no` TINYINT UNSIGNED NOT NULL DEFAULT 1
    COMMENT '时段序号; 同一天多个时段递增',
  `session_type` VARCHAR(16) NOT NULL DEFAULT 'OPEN'
    COMMENT '静态时段类型: PRE_OPEN/OPEN/PRE_CLOSE/CLOSE_ONLY/AUCTION/POST_CLOSE',
  `open_min` SMALLINT NOT NULL
    COMMENT '开市时间(分钟); 基于UTC 00:00; 范围[0,1440]',
  `close_min` SMALLINT NOT NULL
    COMMENT '闭市时间(分钟); 基于UTC 00:00; 范围(open_min,2880]; 允许跨日',
  `fix_session_id` VARCHAR(32) DEFAULT NULL
    COMMENT 'FIX 网关 TradingSessionID; NULL=不暴露给FIX',
  `description` VARCHAR(256) DEFAULT NULL,
  `status` TINYINT UNSIGNED NOT NULL DEFAULT 1,
  `created_at` DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  `updated_at` DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3)
    ON UPDATE CURRENT_TIMESTAMP(3),
  PRIMARY KEY (`id`),
  UNIQUE KEY `uk_day_seq` (`week_day`, `seq_no`),
  CONSTRAINT `chk_session_type_static`
    CHECK (`session_type` IN ('PRE_OPEN', 'OPEN', 'PRE_CLOSE', 'CLOSE_ONLY', 'AUCTION', 'POST_CLOSE')),
  CONSTRAINT `chk_close_after_open` CHECK (`close_min` > `open_min` AND `close_min` <= 2880)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
COMMENT='平台每周交易时段';
```

HALT / MAINTENANCE / QUOTE_ONLY 属于运行时状态，禁止出现在此表 session_type 中（数据库 CHECK 约束兜底）。

5.2 交易日判断（运行时内存计算）

不设独立平台交易日历表：

- 信息冗余：**is_trading_day 可由 t_platform_session（weekday 有无时段）+ t_special_session（有无覆盖）+ t_session_override（有无平台级 HALT）推导
- 粒度不足：**平台级“开 / 不开”过于粗糙，实际判断必须落到品种级 t_daily_session
- 内存更快：**启动时把 t_platform_session 加载为 BitSet[7]，O(1) 判断

运行时判断流程：

启动：

- ① 加载 t_platform_session → 内存 BitSet[7]：哪几天有平台时段
- ② 加载未来 N 日 t_special_session → 按日期索引缓存
- ③ 加载 ACTIVE t_session_override → 按 target 索引缓存

判断“某品种某日是否可交易”：

- ① 直接查 t_daily_session(symbol_id, trading_date) → 若有 state=ACTIVE/SCHEDULED 行 → 可交易
- ② 叠加 t_session_override → 取最严格 runtime_state

前端展示全年交易日历由后端聚合 t_daily_session 生成，数据准确且含品种级细节。

六、品种交易时段

6.1 表结构

```

CREATE TABLE `t_symbol_session` (
  `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  `symbol_id` BIGINT UNSIGNED NOT NULL,
  `week_day` TINYINT UNSIGNED NOT NULL
    COMMENT '星期几(1=周一 7=周日)',
  `seq_no` TINYINT UNSIGNED NOT NULL DEFAULT 1,
  `session_type` VARCHAR(16) NOT NULL DEFAULT 'OPEN'
    COMMENT '静态时段类型; 取值同 t_platform_session',
  `open_min` SMALLINT NOT NULL
    COMMENT '开市时间(分钟); 基于UTC 00:00; 范围[0,1440]',
  `close_min` SMALLINT NOT NULL
    COMMENT '闭市时间(分钟); 基于UTC 00:00; 范围(open_min,2880]; 允许跨日',
  `dst_profile` VARCHAR(16) NOT NULL DEFAULT 'NONE'
    COMMENT 'DST归属: NONE/USA/EUROPE/AUSTRALIA/CHILE/NZ; 系统据此自动生成切换版本',
  `effective_from` DATE NOT NULL DEFAULT '1970-01-01'
    COMMENT '生效起始日期(含)',
  `effective_to` DATE DEFAULT NULL
    COMMENT '生效截止日期(不含); NULL=永久',
  `version` INT UNSIGNED NOT NULL DEFAULT 1
    COMMENT '配置版本号; 同symbol+weekday+seq_no的多版本通过effective区间隔离',
  `source` VARCHAR(16) NOT NULL DEFAULT 'MANUAL'
    COMMENT '来源: MANUAL-人工 / DST_AUTO-系统DST自动生成 / IMPORT-批量导入',
  `status` TINYINT UNSIGNED NOT NULL DEFAULT 1,
  `created_at` DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  `updated_at` DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3)
    ON UPDATE CURRENT_TIMESTAMP(3),
  PRIMARY KEY (`id`),
  UNIQUE KEY `uk_symbol_day_seq_ver` (`symbol_id`, `week_day`, `seq_no`, `version`),
  KEY `idx_symbol_effective` (`symbol_id`, `effective_from`, `effective_to`),
  KEY `idx_dst_profile` (`dst_profile`),
  CONSTRAINT `chk_symbol_session_type`
    CHECK (`session_type` IN ('PRE_OPEN', 'OPEN', 'PRE_CLOSE', 'CLOSE_ONLY', 'AUCTION', 'POST_CLOSE')),
  CONSTRAINT `chk_symbol_close_after_open` CHECK (`close_min` > `open_min` AND `close_min` <= 2880)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
COMMENT='品种每周交易时段';

```

6.2 版本与生效期管理

同 (symbol_id, week_day, seq_no) 多版本共存, 通过 effective_from / effective_to 区间隔离。决胜规则:

```

WHERE effective_from <= trading_date
  AND (effective_to IS NULL OR effective_to > trading_date)
  AND status = 1
ORDER BY version DESC LIMIT 1

```

version 仅作为同区间多次修订的版本号, 不参与时间决胜 (理论上同一 effective_from/to 区间内只应有一条 status=1 的版本)。

6.3 与平台时段的约束

写入校验 (应用层 + 调度任务双重保证):

1. 品种时段不得超出平台时段并集范围
2. 跨日时段在归一化后仍受平台时段约束
3. 同一 (symbol_id, week_day) 内多个 seq_no 不得时段重叠
4. 违反约束的写入返回明确错误码与对应字段, 禁止"静默通过"

6.4 与 DST 的关系

当 `dst_profile != 'NONE'` 时，系统在每年 DST 切换日（由 `t_dst_schedule` 提供）自动生成两条新版本：

- 春令时切换：原版本 `effective_to` 设为切换日，新版本 `effective_from` 设为切换日，`open_min/close_min` 整体提前 60 分钟
- 冬令时切换：反向

详见第十一节。

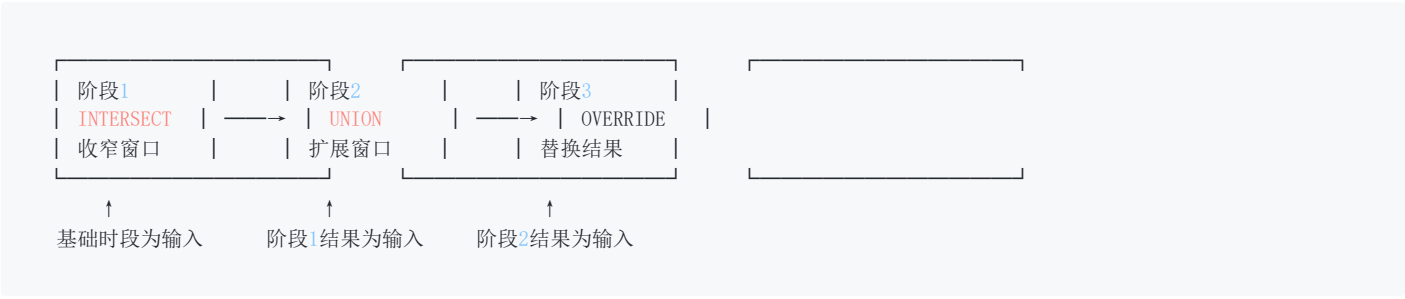
七、特殊日期

7.1 设计目标

替代传统假期表，统一表达“假期休市 / 半天交易 / 临时调整 / 周末加开”等所有非常规日期场景，并通过三阶段管道保证多规则叠加结果的确定性。

7.2 三阶段管道

当多条规则匹配同一品种同一日期时，按固定顺序经三阶段处理：



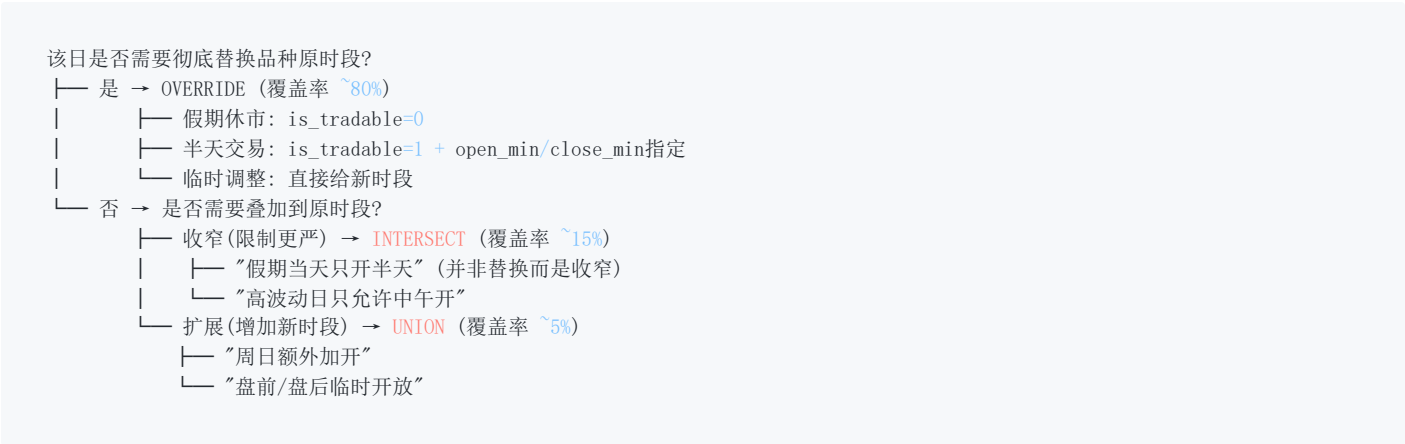
阶段	输入	操作	多规则处理
INTERSECT	品种基础时段	逐条取交集，收窄	<code>open = max(所有open)</code> ， <code>close = min(所有close)</code>
UNION	阶段1 结果	与各 UNION 规则区间合并	排序后合并相邻 / 重叠区间
OVERRIDE	阶段2 结果	取最高优先级规则替换	按 <code>priority DESC</code> , <code>created_at DESC</code> 排序取首条

可交易性 (`is_tradable=0`) 独立于三阶段：在管道执行前判断，任一规则 `is_tradable=0` → 当日整体不可交易。

RELATIVE 模式处理：偏移先与品种当天基础时段合并为绝对时段后进入管道。要求当天基础时段非空。

7.3 stack_policy 使用决策树（实务指引）

实务中 99% 场景应使用 `OVERRIDE`，避免 `INTERSECT/UNION` 混搭的心智负担。决策树如下：



建议：先用 `OVERRIDE` 表达 90% 场景；当确实需要“基于原时段做加减”时才用 `INTERSECT/UNION`。系统在录入时应给出 `OVERRIDE` 默认值与 `stack_policy` 选择的实例化提示。

7.4 特殊日期规则表

SQL

```
CREATE TABLE `t_special_session_rule` (
  `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  `name` VARCHAR(128) NOT NULL,
  `code` VARCHAR(64) NOT NULL
  COMMENT '规则编码; 唯一标识, 用于API引用',
  `path_pattern` VARCHAR(256) NOT NULL
  COMMENT '品种路径匹配模式',
  `priority` SMALLINT UNSIGNED NOT NULL DEFAULT 100
  COMMENT '优先级; 数值越大越优先; 同优先级按created_at DESC',
  `is_tradable` TINYINT UNSIGNED NOT NULL DEFAULT 0
  COMMENT '1=可交易 0=不可交易(否决议)',

  -- 时段指定
  `time_mode` VARCHAR(8) NOT NULL DEFAULT 'ABSOLUTE'
  COMMENT 'ABSOLUTE-绝对分钟数 / RELATIVE-相对常规时段偏移',
  `open_min` SMALLINT NOT NULL DEFAULT 0
  COMMENT 'ABSOLUTE: 开市分钟数 / RELATIVE: 开市偏移',
  `close_min` SMALLINT NOT NULL DEFAULT 1440
  COMMENT 'ABSOLUTE: 闭市分钟数(0~2880) / RELATIVE: 闭市偏移',
  `seq_no` TINYINT UNSIGNED NOT NULL DEFAULT 1,

  -- 递推规则
  `recur_type` VARCHAR(16) NOT NULL
  COMMENT 'FIXED/WEEKDAY_OBSERVED/NTH_WEEKDAY/LAST_WEEKDAY/LUNAR/ISLAMIC/JEWISH/EXPRESSION/MANUAL',
  `month` TINYINT UNSIGNED DEFAULT NULL COMMENT '1-12; FIXED/WEEKDAY_OBSERVED/NTH_WEEKDAY/LAST_WEEKDAY',
  `day` TINYINT UNSIGNED DEFAULT NULL COMMENT '1-31; FIXED/WEEKDAY_OBSERVED',
  `nth_week` TINYINT UNSIGNED DEFAULT NULL COMMENT '1-5; NTH_WEEKDAY',
  `weekday` TINYINT UNSIGNED DEFAULT NULL COMMENT '1-7; NTH_WEEKDAY/LAST_WEEKDAY',
  `lunar_month` TINYINT UNSIGNED DEFAULT NULL COMMENT 'LUNAR/ISLAMIC/JEWISH',
  `lunar_day` TINYINT UNSIGNED DEFAULT NULL,
  `lunar_region` VARCHAR(8) DEFAULT NULL COMMENT 'LUNAR的地区: CN/JP/KR/VN',
  `observed_strategy` VARCHAR(32) DEFAULT 'NONE'
  COMMENT 'WEEKDAY_OBSERVED策略: NONE/FORWARD/BACKWARD/NEAREST/SAT_TO_FRI_SUN_TO_MON',
  `expression` VARCHAR(256) DEFAULT NULL
  COMMENT 'EXPRESSION类型的DSL表达式; 如 "easter+1d" 复活节后一天',

  -- 多日支持
  `duration_days` TINYINT UNSIGNED NOT NULL DEFAULT 1 COMMENT '持续天数',

  -- 叠加策略
  `stack_policy` VARCHAR(16) NOT NULL DEFAULT 'OVERRIDE'
  COMMENT 'OVERRIDE(默认, 99%场景)/INTERSECT(收窄)/UNION(扩展)',

  -- 例外
  `excluded_years` JSON DEFAULT NULL
  COMMENT '跳过的年份数组; 如某年因奥运会取消',

  -- 生效与状态
  `description` VARCHAR(256) DEFAULT NULL,
  `effective_from` YEAR NOT NULL DEFAULT 2020,
  `effective_to` YEAR DEFAULT NULL,
  `status` TINYINT UNSIGNED NOT NULL DEFAULT 1,
  `created_at` DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  `updated_at` DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3)
  ON UPDATE CURRENT_TIMESTAMP(3),
  PRIMARY KEY (`id`),
  UNIQUE KEY `uk_code` (`code`),
  KEY `idx_status_recur` (`status`, `recur_type`),
  CONSTRAINT `chk_rule_close` CHECK (`close_min` BETWEEN -1440 AND 2880),
  CONSTRAINT `chk_rule_stack_policy` CHECK (`stack_policy` IN ('OVERRIDE', 'INTERSECT', 'UNION'))
```



```
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
COMMENT='特殊日期规则';
```

7.5 recur_type 完整定义

recur_type	必填字段	含义	示例
FIXED	month, day	公历固定日期	元旦：1/1
WEEKDAY_OBSERVED	month, day, observed_strategy	公历日期 + 周末顺延策略	美国独立日：7/4， SAT_TO_FRI_SUN_TO_MON
NTH_WEEKDAY	month, nth_week, weekday	某月第 N 个星期几	美国感恩节：11月第4个周四
LAST_WEEKDAY	month, weekday	某月最后一个星期几	美国阵亡将士纪念日：5月最后一个周一
LUNAR	lunar_month, lunar_day, lunar_region	农历日期 + 地区	春节：CN 正月初一
ISLAMIC	lunar_month, lunar_day	伊斯兰历（希吉来历）	开斋节：10月1日（希吉来）
JEWISH	lunar_month, lunar_day	希伯来历	赎罪日
EXPRESSION	expression	DSL 表达式	easter+1d（复活节后一天）
MANUAL	—	不自动递推，手动录入 t_special_session	临时休市

observed_strategy 取值：

策略	含义
NONE	不调整
FORWARD	周末统一顺延至周一
BACKWARD	周末统一回退至周五
NEAREST	周六→周五，周日→周一（美国独立日规则）
SAT_TO_FRI_SUN_TO_MON	NEAREST 的明确别名

7.6 特殊日期实例表

```
SQL

CREATE TABLE `t_special_session` (
  `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  `name` VARCHAR(128) NOT NULL,
  `date` DATE NOT NULL,
  `path_pattern` VARCHAR(256) NOT NULL,
  `priority` SMALLINT UNSIGNED NOT NULL DEFAULT 100,
  `is_tradable` TINYINT UNSIGNED NOT NULL DEFAULT 0,
  `time_mode` VARCHAR(8) NOT NULL DEFAULT 'ABSOLUTE',
  `open_min` SMALLINT NOT NULL DEFAULT 0,
  `close_min` SMALLINT NOT NULL DEFAULT 1440,
  `stack_policy` VARCHAR(16) NOT NULL DEFAULT 'OVERRIDE',
  `seq_no` TINYINT UNSIGNED NOT NULL DEFAULT 1,
  `rule_id` BIGINT UNSIGNED DEFAULT NULL
  COMMENT '来源规则ID; NULL=人工录入',
  `rule_version` INT UNSIGNED DEFAULT NULL
)
```

```

        `source`          COMMENT '生成时的规则版本；用于规则变更后的实例同步判断',
                          VARCHAR(16)          NOT NULL DEFAULT 'MANUAL'
        COMMENT 'MANUAL/RULE_AUTO/EXCHANGE_SYNC',
        `description`     VARCHAR(256)          DEFAULT NULL,
        `status`          TINYINT UNSIGNED     NOT NULL DEFAULT 1,
        `created_at`      DATETIME(3)         NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
        `updated_at`      DATETIME(3)         NOT NULL DEFAULT CURRENT_TIMESTAMP(3)
                          ON UPDATE CURRENT_TIMESTAMP(3),
        PRIMARY KEY (`id`),
        UNIQUE KEY `uk_date_pattern_seq_policy` (`date`, `path_pattern`, `seq_no`, `stack_policy`),
        KEY `idx_date_status` (`date`, `status`),
        KEY `idx_pattern_date` (`path_pattern`, `date`),
        KEY `idx_rule_id` (`rule_id`)
    ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
    COMMENT='特殊日期实例';

```

7.7 多规则叠加示例（保留关键场景）

示例1：OVERRIDE（最常见，99% 场景）

某品种常规 09:30-16:00（[570,960]），感恩节后天提前到 13:00 收盘：

规则	is_tradable	stack_policy	open_min	close_min
半天交易	1	OVERRIDE	570	780

→ 结果 [570, 780]（09:30-13:00）。逻辑简单直观。

示例2：UNION（周日额外加开）

EURUSD 周日默认不交易（基础时段空），增加两个加开时段：

规则	stack_policy	open_min	close_min
上午场	UNION	540	600
午间场	UNION	720	840

→ 阶段1 INTERSECT 无输入透传 → 阶段2 UNION 输出 [540,600]，[720,840] → 阶段3 OVERRIDE 无规则透传 → 最终两段独立时段，预计算写入 t_daily_session 两行。

示例3：INTERSECT + UNION 组合

常规 09:30-16:00，假期提前收盘但增加盘前加开：

规则	stack_policy	open_min	close_min
提前收盘	INTERSECT	570	780
盘前加开	UNION	480	570

→ 阶段1 INTERSECT [570, 780] → 阶段2 UNION [480, 780] → 最终 08:00-13:00。

示例4：OVERRIDE 同优先级决胜

两条 OVERRIDE 同 priority=100：取 created_at 最新者生效（Last-Write-Wins）。

7.8 规则变更与实例同步

当 t_special_session_rule 修改后，已生成的 t_special_session 实例处理策略：

修改类型	同步策略
优先级 / 时段 / 可交易性	标记受影响实例 status=0，重新由规则引擎生成新实例

修改类型	同步策略
递推规则（month/day/recur_type）	删除该 rule_id 名下所有未来日期实例，重新递推
excluded_years 新增年份	删除对应年份实例
effective_from/to 缩短	删除超出范围的实例
人工实例（rule_id IS NULL）	不受影响

实例上保留 rule_version 用于变更后的批量识别。

八、每日交易时段快照

8.1 表结构（含分区）

```
SQL

CREATE TABLE `t_daily_session` (
  `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  `symbol_id` BIGINT UNSIGNED NOT NULL,
  `trading_date` DATE NOT NULL,
  `seq_no` TINYINT UNSIGNED NOT NULL DEFAULT 1,
  `session_type` VARCHAR(16) NOT NULL
  COMMENT '静态时段类型: PRE_OPEN/OPEN/PRE_CLOSE/CLOSE_ONLY/AUCTION/POST_CLOSE',
  `open_min` SMALLINT NOT NULL
  COMMENT '开市分钟数(UTC); 范围[0, 1440]',
  `close_min` SMALLINT NOT NULL
  COMMENT '闭市分钟数(UTC); 范围(open_min, 2880]; 允许跨日',
  `state` VARCHAR(16) NOT NULL DEFAULT 'SCHEDULED'
  COMMENT 'SCHEDULED/ACTIVE/COMPLETED/CANCELLED',
  `runtime_state` VARCHAR(16) DEFAULT NULL
  COMMENT '运行时状态: HALT/MAINTENANCE/QUOTE_ONLY; NULL=无覆盖',
  `override_id` BIGINT UNSIGNED DEFAULT NULL
  COMMENT '当前生效的覆盖ID; runtime_state非NULL时必须填',
  `source_type` VARCHAR(16) NOT NULL DEFAULT 'NORMAL'
  COMMENT 'NORMAL/SPECIAL_DATE/OVERRIDE',
  `source_id` BIGINT UNSIGNED DEFAULT NULL,
  `generated_at` DATETIME(3) NOT NULL,
  `created_at` DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  `updated_at` DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3)
  ON UPDATE CURRENT_TIMESTAMP(3),
  PRIMARY KEY (`id`, `trading_date`),
  UNIQUE KEY `uk_symbol_date_seq` (`symbol_id`, `trading_date`, `seq_no`),
  KEY `idx_date_state` (`trading_date`, `state`),
  KEY `idx_symbol_date` (`symbol_id`, `trading_date`),
  KEY `idx_runtime_state` (`runtime_state`, `trading_date`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
COMMENT='每日交易时段快照'
PARTITION BY RANGE (TO_DAYS(`trading_date`)) (
  PARTITION p_init VALUES LESS THAN (TO_DAYS('2026-01-01')),
  PARTITION p_2026q1 VALUES LESS THAN (TO_DAYS('2026-04-01')),
  PARTITION p_2026q2 VALUES LESS THAN (TO_DAYS('2026-07-01')),
  PARTITION p_2026q3 VALUES LESS THAN (TO_DAYS('2026-10-01')),
  PARTITION p_2026q4 VALUES LESS THAN (TO_DAYS('2027-01-01')),
  PARTITION p_future VALUES LESS THAN MAXVALUE
);
```

分区策略：按 trading_date RANGE 季度分区，调度任务每季度初动态新增分区，并将 p_future 滚动后移。归档时直接 ALTER TABLE ... DROP PARTITION，秒级完成。

8.2 预计算流程

每日 00:00 UTC 调度任务：

① 加载 t_dst_schedule 当日是否切换 → 若是，触发受影响品种 t_symbol_session 版本切换

② 查 t_platform_session → 当日平台时段

③ 查 t_symbol_session → 各品种常规时段(取生效版本)

④ 查 t_special_session → 当日特殊日期实例

a. 可交易性判断：任一 is_tradable=0 → 当日整体不可交易

b. RELATIVE → ABSOLUTE 转换

c. 三阶段管道叠加：INTERSECT → UNION → OVERRIDE

d. 路径匹配在 Trie 内完成, 运行时零路径解析

⑤ 合并写入 t_daily_session → state=SCHEDULED

⑥ 写运行日志 t_snapshot_run_log

⑦ 失败重试：最多3次，失败告警，保留上次快照

8.3 状态机

SCHEDULED → ACTIVE → COMPLETED

↓

CANCELLED（因实时覆盖等）

状态	含义	触发
SCHEDULED	已排期，未到开始时间	预计算写入
ACTIVE	当前正在交易	now 进入 [open_min, close_min)
COMPLETED	已正常结束	now ≥ close_min
CANCELLED	因紧急事件取消	实时覆盖 HALT 等

runtime_state 与 state 解耦：state 描述时段生命周期，runtime_state 描述当前是否被覆盖到非常规状态。

8.4 查询性能对比

场景	无快照	有快照
下单可否交易	多表 JOIN + 路径匹配 + 时段计算	一条 SELECT + runtime_state 检查
前端展示今日时段	复杂计算	一条 SELECT
规则变更影响评估	回溯历史计算	重生成受影响日期快照
性能	规则越复杂越慢	O(1) + 索引命中

九、实时覆盖

9.1 设计原则

1. 覆盖即状态切换：每条覆盖将目标的 runtime_state 切换为指定运行时状态，与第三节运行时类型语义统一
2. 解除即关闭记录：暂停的解除不新建 RESUME 记录，而是更新原记录 status=LIFTED + lifted_by/lifted_at/lift_reason，保持完整事件链
3. 强制平仓不属于此表：FORCE_CLOSE 为一次性风控操作，由风控系统独立处理
4. 层级覆盖运行时评估：不创建子记录，按 PLATFORM → EXCHANGE → CATEGORY → PRODUCT 评估取最严格
5. 分阶段恢复：支持 NYSE 竞价恢复、Binance 多阶段放开等渐进恢复模式

6. 幂等性：通过 idempotency_key 防重复触发（交易所推送重试 / 风控规则重复触发等）
7. 合规可追溯：内建 disclosed_at / public_announcement_id / approver 满足 MiFID II Article 49 公开披露要求

9.2 真实场景全景

交易所发起

场景	范围	持续	恢复	runtime_state
NYSE L1/L2 熔断	全市场	15min	自动竞价恢复	HALT → AUCTION → 解除
NYSE L3 熔断	全市场	当日剩余	不恢复	HALT
LULD 个股波动	单品种	5s-10min	自动竞价	HALT → AUCTION
SEC 强制暂停	单品种	~10 交易日	手动	HALT

经纪商 / 平台发起

场景	范围	持续	恢复	runtime_state
风控紧急暂停	品种/分类	不定	手动	HALT 或 CLOSE_ONLY
系统故障	平台级	不定	分阶段	HALT → QUOTE_ONLY → CLOSE_ONLY → 解除
计划维护	平台级	预设	手动/自动	MAINTENANCE
保证金调整	品种/分类	持续	手动	CLOSE_ONLY
极端行情限制开仓	品种/分类	不定	手动	CLOSE_ONLY

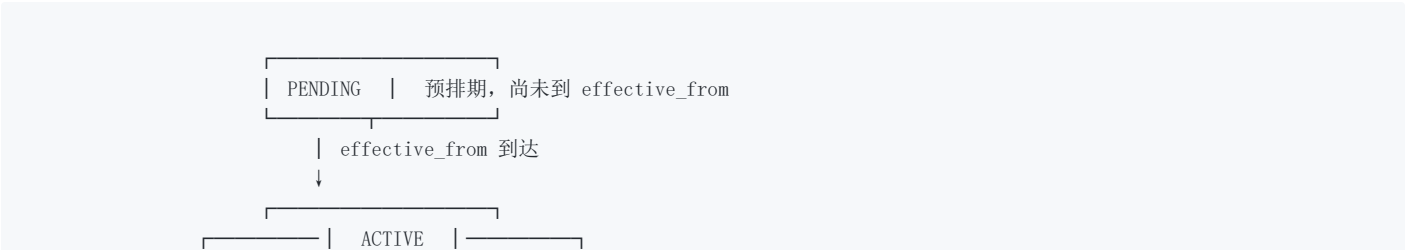
监管发起

场景	范围	持续	恢复	runtime_state
MiFID II 市场暂停	跨场所	不定	监管决定	HALT
监管审查	单品种	10-20 交易日	监管决定	HALT
紧急市场关闭	全市场	不定	政府决定	HALT

产品事件

场景	范围	处理	runtime_state
合约到期	单品种	切换	CLOSE_ONLY → HALT
合约换月	旧/新	自动	旧 CLOSE_ONLY→HALT；新 PRE_OPEN→OPEN
品种退市	单品种	永久	CLOSE_ONLY → HALT
公司行为	单品种	短暂	HALT → 解除

9.3 状态机





特殊路径：
PENDING → CANCELLED 排期后、生效前取消
ACTIVE → SUPERSEDED 被同 target 新覆盖替换

9.4 覆盖表

```
SQL
CREATE TABLE `t_session_override` (
  `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,

  -- 幂等性
  `idempotency_key` VARCHAR(128) NOT NULL
  COMMENT '幂等键; 同key重复请求返回原记录; 推荐格式 {source}:{source_ref}:{action}',

  -- 覆盖目标
  `target_kind` VARCHAR(16) NOT NULL
  COMMENT 'PLATFORM/EXCHANGE/CATEGORY/PRODUCT',
  `target_value` VARCHAR(256) NOT NULL
  COMMENT '目标值; PLATFORM="ALL"; EXCHANGE=MIC码; CATEGORY=path_pattern; PRODUCT=symbol_id字符串',

  -- 覆盖行为
  `override_session_type` VARCHAR(16) NOT NULL
  COMMENT '运行时状态: HALT/CLOSE_ONLY/QUOTE_ONLY/PRE_OPEN/AUCTION/MAINTENANCE',
  `severity` TINYINT UNSIGNED NOT NULL DEFAULT 2
  COMMENT '1=低(计划维护) 2=中(风控限制) 3=高(交易所熔断) 4=紧急(系统故障/监管)',

  -- 生效与恢复
  `effective_from` DATETIME(3) NOT NULL
  COMMENT '生效时间(UTC); 可为未来时间(预排期)',
  `auto_resume_time` DATETIME(3) DEFAULT NULL
  COMMENT '自动恢复时间(UTC); NULL=持续到手动解除',

  -- 来源与审计
  `source` VARCHAR(16) NOT NULL DEFAULT 'OPS'
  COMMENT 'EXCHANGE/RISK/OPS/REGULATOR/SYSTEM',
  `source_ref` VARCHAR(256) DEFAULT NULL
  COMMENT '来源引用: 交易所暂停代码/风控规则ID/工单号/监管文号',
  `reason` VARCHAR(512) NOT NULL,
  `operator` BIGINT UNSIGNED NOT NULL
  COMMENT '操作人ID; SYSTEM时为系统账户ID',
  `approver` BIGINT UNSIGNED DEFAULT NULL
  COMMENT '审批人ID; 高 severity 强制要求双人审批',

  -- 分阶段恢复
  `resume_phase` VARCHAR(16) NOT NULL DEFAULT 'IMMEDIATE'
  COMMENT 'IMMEDIATE/QUOTE_FIRST/AUCTION_FIRST/PHASED',
  `resume_phases` JSON DEFAULT NULL
  COMMENT 'PHASED 模式阶段定义',
  `previous_phase_id` BIGINT UNSIGNED DEFAULT NULL
  COMMENT 'PHASED 链路上上一阶段ID',

  -- 状态与解除
```

```
`status`                VARCHAR(16)                NOT NULL DEFAULT 'PENDING'
                        COMMENT 'PENDING/ACTIVE/LIFTED/EXPIRED/SUPERSEDED/CANCELLED',
`lifted_by`             BIGINT UNSIGNED                DEFAULT NULL,
`lifted_at`             DATETIME(3)                    DEFAULT NULL,
`lift_reason`           VARCHAR(256)                   DEFAULT NULL,
`superseded_by`         BIGINT UNSIGNED                DEFAULT NULL,

-- 合规披露 (MiFID II Article 49)
`disclosed_at`          DATETIME(3)                    DEFAULT NULL
                        COMMENT '公开披露时间; 监管要求 severity>=3 必填',
`public_announcement_id` VARCHAR(128)                  DEFAULT NULL
                        COMMENT '公告ID; 关联客户公告系统',

`created_at`            DATETIME(3)                    NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
`updated_at`            DATETIME(3)                    NOT NULL DEFAULT CURRENT_TIMESTAMP(3)
                        ON UPDATE CURRENT_TIMESTAMP(3),

PRIMARY KEY (`id`),
UNIQUE KEY `uk_idempotency` (`idempotency_key`),
UNIQUE KEY `uk_active_target` (`target_kind`, `target_value`, `status`)
    COMMENT '同一target同时只能有一条ACTIVE覆盖',
KEY `idx_status_effective` (`status`, `effective_from`),
KEY `idx_auto_resume` (`status`, `auto_resume_time`),
KEY `idx_source` (`source`, `created_at`),
CONSTRAINT `chk_override_session_type`
    CHECK (`override_session_type` IN ('HALT', 'CLOSE_ONLY', 'QUOTE_ONLY', 'PRE_OPEN', 'AUCTION', 'MAINTENANCE'))
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
COMMENT='交易时段实时覆盖';
```

9.5 target_kind 与目标规范化

target_kind	target_value 取值	示例
PLATFORM	固定字符串 "ALL"	"ALL"
EXCHANGE	MIC 码 (ISO 10383)	"XNYS", "XNAS", "XHKG"
CATEGORY	path_pattern	"Crypto/**", "Stocks/US/*"
PRODUCT	symbol_id 字符串形式	"12345"

应用层强制校验 target_value 格式与 target_kind 匹配，避免脏数据。

9.6 override_session_type 定义

类型	含义	典型场景	交易行为
HALT	交易暂停	熔断/监管/新闻	不可下单 / 不可挂单 / 现有挂单按配置保留或取消
CLOSE_ONLY	仅平仓	保证金/极端行情/到期	可平不可开
QUOTE_ONLY	仅行情	恢复初期/盘前查看	可看不可下单
PRE_OPEN	盘前	恢复过渡/新品种	可挂单不可成交
AUCTION	集合竞价	熔断恢复	竞价撮合
MAINTENANCE	系统维护	计划维护	不可下单 / 不可挂单

HALT vs MAINTENANCE 区别：交易行为相同，但语义不同——HALT 紧急（交易所/风控/监管），需全渠道告警；MAINTENANCE 计划（运维），仅内部通知。severity 字段进一步区分同类内紧急程度。

9.7 层级覆盖与级联

target_kind 层级(从高到低):

- PLATFORM → 全平台所有品种
- EXCHANGE → 指定交易所下所有品种
- CATEGORY → 指定分类路径下所有品种
- PRODUCT → 指定品种

级联规则:

- 1. 高层级覆盖生效时, 自动级联到所有低层级 target
- 2. 同 (target_kind, target_value) 同时只能有一条 ACTIVE (UNIQUE 约束)
- 3. 低层级覆盖不覆盖高层级: PLATFORM HALT 存在时, PRODUCT 级写入应被拒绝 (或先解除 PLATFORM 再写 PRODUCT)
- 4. 高层级解除时, 低层级品种恢复各自原状态

运行时内存评估 (微秒级):

品种实际状态 = 最严格 (

- PLATFORM ACTIVE override,
- 所属交易所 ACTIVE override,
- 所属各分类 ACTIVE override,
- 自身 ACTIVE override

)

严格度: HALT > CLOSE_ONLY > MAINTENANCE > QUOTE_ONLY > PRE_OPEN > AUCTION

9.8 分阶段恢复 (PHASED)

resume_phase 取值:

策略	含义	场景
IMMEDIATE	直接恢复	简单风控暂停
QUOTE_FIRST	先行情再交易	系统故障恢复
AUCTION_FIRST	先竞价再连续交易	NYSE 熔断恢复
PHASED	自定义分阶段	平台级、Binance 式

resume_phases 使用相对时长 JSON:

```
JSON

[
  { "phase": "QUOTE_ONLY", "duration_min": 10 },
  { "phase": "CLOSE_ONLY", "duration_min": 15 },
  { "phase": "OPEN", "duration_min": 0 }
]
```

PHASED 执行链路: 每个阶段创建新 override 记录, previous_phase_id 关联上一阶段。任一阶段可被手动解除 (status=LIFTED) 跳过后续。

冲突处理: PHASED 链路执行过程中出现新覆盖请求时:

- 新覆盖严重度 ≤ 当前阶段 → 拒绝
- 新覆盖严重度 > 当前阶段 → 中断 PHASED 链路, 新覆盖直接生效
- 链路中断后, 恢复需重新发起 PHASED

9.9 替换机制 (SUPERSEDED)

品种X已有 ACTIVE CLOSE_ONLY (id=100)
→ 风控决定完全暂停
处理：
① 旧 id=100: status=SUPERSEDED, superseded_by=101
② 新 id=101: override_session_type=HALT, status=ACTIVE

允许降级（HALT→CLOSE_ONLY），由操作人 reason 解释。所有历史记录保留（监管审计）。

9.10 与 auto_resume_time 的交互

场景	auto_resume_time	resume_phase	行为
无限期暂停	NULL	IMMEDIATE	持续到手动解除
定时自动恢复	非 NULL	IMMEDIATE	到时间自动恢复
熔断竞价恢复	非 NULL	AUCTION_FIRST	到时间后进入 AUCTION
自定义分阶段	NULL	PHASED	运维启动逐步推进
定时 + 分阶段	非 NULL	PHASED	到时间自动启动 PHASED

手动恢复优先级高于自动恢复。

9.11 常见场景示例

场景 1：NYSE L1 熔断（自动恢复 + 竞价过渡）

插入 override:

```
target_kind=EXCHANGE, target_value='XNYS',
override_session_type=HALT,
source=EXCHANGE, source_ref='NYSE-MWCB-L1-20260520',
idempotency_key='EXCHANGE:NYSE-MWCB-L1-20260520:HALT',
severity=3, auto_resume_time=NOW()+15min,
resume_phase=AUCTION_FIRST, status=ACTIVE,
disclosed_at=NOW(), public_announcement_id='ANN-20260520-001'
```

场景 2：风控紧急暂停（手动解除）

插入 override:

```
target_kind=PRODUCT, target_value='12345',
override_session_type=CLOSE_ONLY,
source=RISK, source_ref='RULE-ABNORMAL-VOL-001',
idempotency_key='RISK:RULE-ABNORMAL-VOL-001:12345:CLOSE_ONLY:20260520-T1430',
severity=2, auto_resume_time=NULL, status=ACTIVE
```

解除:

```
UPDATE t_session_override
SET status='LIFTED', lifted_by=1001, lifted_at=NOW(3),
    lift_reason='波动回归正常'
WHERE id=...
```

场景 3：平台级紧急暂停（分阶段恢复）

插入 override:

```
target_kind=PLATFORM, target_value='ALL',
override_session_type=HALT,
```

```
source=OPS, severity=4, reason=' 核心交易引擎异常',
approver=2001,
idempotency_key=' OPS:INCIDENT-20260520:PLATFORM-HALT',
resume_phase=PHASED,
resume_phases=[
    {"phase": "QUOTE_ONLY", "duration_min": 10},
    {"phase": "PRE_OPEN", "duration_min": 5},
    {"phase": "CLOSE_ONLY", "duration_min": 15},
    {"phase": "OPEN", "duration_min": 0}
],
disclosed_at=NOW(), public_announcement_id=' ANN-20260520-INCIDENT'
```

十、交易所日历数据源

10.1 定位

交易所日历数据是本系统的外部数据源而非计算引擎：

- 1. 数据源而非计算引擎：最终时段仍由 t_special_session + 三阶段管道决定
- 2. 参考 + 辅助录入：运营据此生成 t_special_session 实例
- 3. 独立于核心链路：即使此表缺失，运营手动维护 t_special_session 仍可正常运行

10.2 数据源现状

数据源	类型	覆盖	可靠性
交易所官网	网页/PDF	单交易所	官方权威，需手动录入
JPX J-Quants API	REST	东京交易所	高
HKEX iCal	CalDAV	香港交易所	高
EODHD API	REST	70+ 交易所	高，\$49-\$249/月
Nasdaq Data Link EFPH	REST	2,000+ 机构	高，企业级
pandas_market_calendars	Python	60+ 交易所	中，开源
Finnhub	REST	主要交易所	中，免费额度 60/min
Bloomberg/Refinitiv	专有	全球	极高，~\$22,000/年

关键事实：多数交易所不提供官方机器可读 API；交易所偶尔临时调整（极端天气、国丧）；不同源数据可能冲突，需以运营确认为准。

10.3 交易所假日表

```
SQL

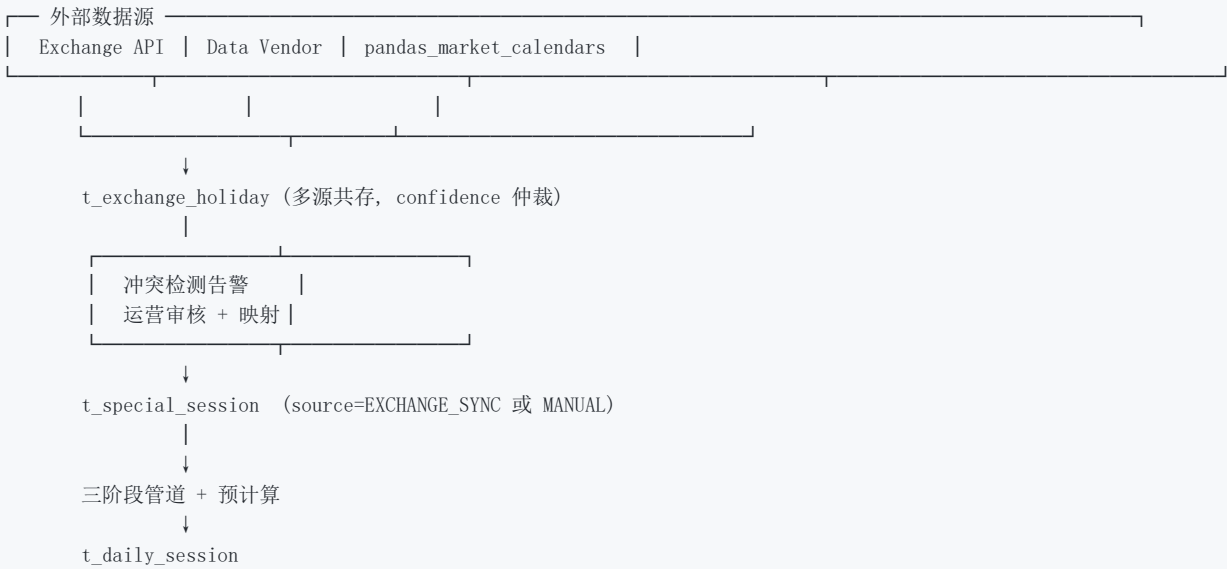
CREATE TABLE `t_exchange_holiday` (
  `id`          BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  `exchange_code` VARCHAR(16) NOT NULL
  COMMENT 'MIC码(ISO 10383): XNYS/XNAS/XLON/XJPX/XHKG/XSHG/XSHE',
  `date`       DATE NOT NULL,
  `holiday_name` VARCHAR(128) DEFAULT NULL,
  `is_trading_day` TINYINT UNSIGNED NOT NULL DEFAULT 1,
  `day_session_type` VARCHAR(16) NOT NULL DEFAULT 'FULL'
  COMMENT 'FULL/EARLY_CLOSE/LATE_OPEN/CLOSED',
  `open_min`    SMALLINT DEFAULT NULL
  COMMENT ' 开市分钟数(UTC); NULL=用默认',
  `close_min`   SMALLINT DEFAULT NULL
```

```

        COMMENT ' 闭市分钟数(UTC); NULL=用默认',
`year`          SMALLINT UNSIGNED  NOT NULL,
`source`        VARCHAR(32)        NOT NULL DEFAULT 'MANUAL'
        COMMENT 'MANUAL/EXCHANGE_API/EODHD/PANDAS_MC/...',
`source_ref`    VARCHAR(256)       DEFAULT NULL,
`confidence`    VARCHAR(8)         NOT NULL DEFAULT 'HIGH'
        COMMENT 'HIGH/MEDIUM/LOW; 用于多源冲突时仲裁',
`superseded_by` BIGINT UNSIGNED    DEFAULT NULL
        COMMENT '被哪条记录替换; 同(exchange,date)新数据来时旧数据置位',
`synced_at`     DATETIME(3)        DEFAULT NULL,
`status`        TINYINT UNSIGNED   NOT NULL DEFAULT 1,
`created_at`    DATETIME(3)        NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
`updated_at`    DATETIME(3)        NOT NULL DEFAULT CURRENT_TIMESTAMP(3)
        ON UPDATE CURRENT_TIMESTAMP(3),
PRIMARY KEY (`id`),
UNIQUE KEY `uk_exchange_date_source` (`exchange_code`, `date`, `source`)
        COMMENT '允许多源数据共存; 冲突时由confidence仲裁',
KEY `idx_year_status` (`year`, `status`),
KEY `idx_exchange_date` (`exchange_code`, `date`, `status`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
COMMENT='交易所假日日历';

```

10.4 数据流：交易所日历 → 特殊日期



10.5 冲突检测与仲裁

当多个数据源对同 (exchange_code, date) 给出不同数据时：

- 系统按 confidence DESC, synced_at DESC 取最高置信度数据为"主数据"
- 若不同源数据的 is_trading_day 或 day_session_type 不一致 → 触发告警，运营手动裁定
- 运营裁定后，被采纳的记录 status=1 其余 status=0 并写 superseded_by
- 主数据生成 t_special_session 实例时 source=EXCHANGE_SYNC，记录 source_ref 指向 t_exchange_holiday.id

10.6 同步策略

策略	数据源	频率	场景
手动录入	官网	每年初	所有交易所基准确认

策略	数据源	频率	场景
API 同步	JPX/HKEX	每周	有官方 API 的交易所
数据供应商	EODHD/Nasdaq EFPH	每日	高覆盖付费
开源库初始化	pandas_market_calendars	首次/季度	快速启动零成本

十一、DST 处理

11.1 设计目标

MT5 通过本地时间 + DST profile 自动处理夏令时切换，无需运营每年为每个品种手动新增两个版本。本设计在保持"配置层全 UTC"基线的前提下，引入 `dst_profile` + `t_dst_schedule` + 自动版本生成器，达成同样效果。

11.2 DST profile 定义

profile	春令时(春进)	冬令时(秋退)	适用
NONE	—	—	无 DST，如中国大陆、日本、香港、新加坡
USA	3月第2个周日 02:00→03:00	11月第1个周日 02:00→01:00	美国、加拿大大部分
EUROPE	3月最后周日 01:00 UTC→02:00 UTC	10月最后周日 01:00 UTC→00:00 UTC	欧盟（注：2021 起欧盟讨论取消 DST，但尚未实施）
AUSTRALIA	10月第1个周日 02:00→03:00	4月第1个周日 03:00→02:00	澳大利亚东南部
CHILE	9月第1个周六午夜	4月第1个周六午夜	智利
NZ	9月最后周日 02:00→03:00	4月第1个周日 03:00→02:00	新西兰

DST profile 的具体切换日由 `t_dst_schedule` 表枚举（预生成未来 5 年），避免运行时计算复杂日历规则。

11.3 t_dst_schedule 表

SQL

```
CREATE TABLE `t_dst_schedule` (
  `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  `dst_profile` VARCHAR(16) NOT NULL
    COMMENT 'USA/EUROPE/AUSTRALIA/CHILE/NZ',
  `year` SMALLINT UNSIGNED NOT NULL,
  `spring_forward_date` DATE NOT NULL
    COMMENT '春进日期(UTC计); 当地2:00→3:00',
  `fall_back_date` DATE NOT NULL
    COMMENT '秋退日期(UTC计); 当地2:00→1:00',
  `utc_offset_dst` SMALLINT NOT NULL
    COMMENT '夏令时UTC偏移(分钟); 如美东EDT=-240',
  `utc_offset_std` SMALLINT NOT NULL
    COMMENT '标准时UTC偏移(分钟); 如美东EST=-300',
  `status` TINYINT UNSIGNED NOT NULL DEFAULT 1,
  `created_at` DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  `updated_at` DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3)
    ON UPDATE CURRENT_TIMESTAMP(3),
  PRIMARY KEY (`id`),
  UNIQUE KEY `uk_profile_year` (`dst_profile`, `year`),
  KEY `idx_spring_forward` (`spring_forward_date`),
  KEY `idx_fall_back` (`fall_back_date`)
```

```
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
COMMENT='夏令时切换日历';
```

11.4 自动版本生成流程

每年 12 月（或运营手动触发）执行 DST 版本生成器：

```
For each profile in (USA, EUROPE, AUSTRALIA, CHILE, NZ):
① 查 t_dst_schedule(profile, year+1) → 获取 spring_forward_date / fall_back_date
② 查所有 t_symbol_session WHERE dst_profile=profile AND effective_to IS NULL
③ 对每条 session 生成两条新版本：
    a. 春令时版本：
        - 原版本：effective_to = spring_forward_date
        - 新版本：effective_from = spring_forward_date
            open_min/close_min 整体 -60 (UTC 看, 品种本地时间不变)
            source='DST_AUTO', version+=1
    b. 冬令时版本：
        - 春令时版本：effective_to = fall_back_date
        - 冬令时版本：effective_from = fall_back_date
            open_min/close_min 整体 +60 (复原)
            source='DST_AUTO', version+=1
④ 写 t_snapshot_run_log 记录生成数量
```

11.5 切换时刻校验

DST 切换日的预计算需要特别注意：

1. **春进日**当地凌晨 02:00 跳到 03:00，意味着该日“少了一小时”。需检查跨日时段是否落在不存在的小时内
2. **秋退日**当地 02:00 跳到 01:00，意味着该日“多了一小时”，01:00~02:00 出现两次。需明确订单时间归属
3. UTC 视角下两个切换日均为正常 24 小时，无歧义。所以**应用层和运行时所有逻辑都在 UTC 下进行**，仅前端展示时按品种 primary_timezone 转本地时间

十二、配置变更审计

12.1 设计目标

满足 MiFID II Article 50 与 SEC 17a-4 对配置变更的完整可追溯要求：变更前后快照、变更原因、变更来源、操作 IP、双人审批。

12.2 表结构

SQL

```
CREATE TABLE `t_config_audit` (
  `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  `table_name` VARCHAR(64) NOT NULL,
  `record_id` BIGINT UNSIGNED NOT NULL,
  `action` VARCHAR(16) NOT NULL,
  COMMENT 'INSERT/UPDATE/DELETE/RESTORE',
  `before_data` JSON DEFAULT NULL,
  `after_data` JSON DEFAULT NULL,
  `reason` VARCHAR(512) DEFAULT NULL,
  COMMENT '变更原因; 推荐关联工单号',
  `request_id` VARCHAR(64) DEFAULT NULL,
  COMMENT '请求追踪ID; 与 trace_id 联动',
  `change_source` VARCHAR(16) NOT NULL DEFAULT 'UI'
  COMMENT 'UI/API/SCRIPT/SYSTEM/DST_AUTO',
  `operator` BIGINT UNSIGNED NOT NULL,
  `approver` BIGINT UNSIGNED DEFAULT NULL
```

```

        COMMENT '审批人ID; 关键配置变更强制双人审批',
`client_ip`          VARCHAR(64)          DEFAULT NULL,
`user_agent`         VARCHAR(256)         DEFAULT NULL,
`created_at`         DATETIME(3)          NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
PRIMARY KEY (`id`),
KEY `idx_table_record` (`table_name`, `record_id`),
KEY `idx_operator_created` (`operator`, `created_at`),
KEY `idx_request` (`request_id`),
KEY `idx_created` (`created_at`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
COMMENT='配置变更审计';

```

12.3 双人审批清单

以下变更强制要求 approver IS NOT NULL ，由应用层校验：

表	变更类型	理由
t_platform_session	任意变更	平台级影响全市场
t_session_override	severity>=3 创建/解除	监管/紧急事件需审批
t_special_session_rule	大规模 path_pattern 规则	影响面大
t_dst_schedule	手动修改（非自动生成）	直接影响 DST 自动版本

十三、运行状态与可观测性

13.1 t_snapshot_run_log 预计算运行日志

```

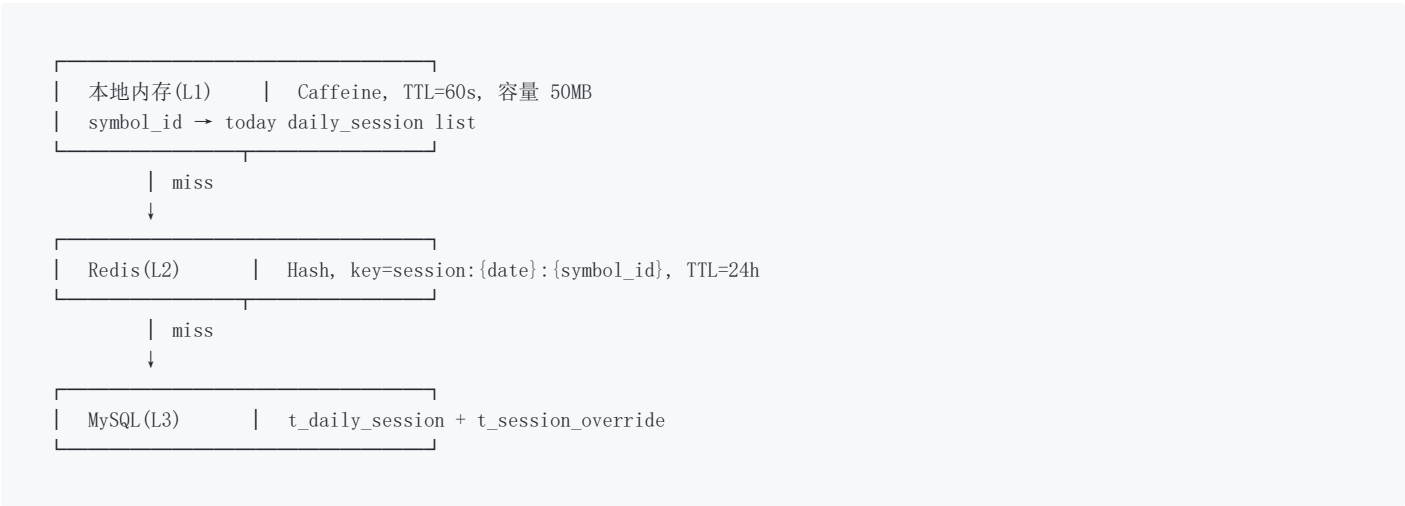
SQL

CREATE TABLE `t_snapshot_run_log` (
  `id`          BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  `run_id`      VARCHAR(64)     NOT NULL
  COMMENT '运行ID; UUID',
  `run_type`    VARCHAR(16)     NOT NULL
  COMMENT 'DAILY/MANUAL/REPAIR/DST_VERSION_GEN',
  `target_date` DATE           NOT NULL
  COMMENT '生成的快照日期',
  `started_at`  DATETIME(3)     NOT NULL,
  `finished_at` DATETIME(3)     DEFAULT NULL,
  `status`     VARCHAR(16)     NOT NULL DEFAULT 'RUNNING'
  COMMENT 'RUNNING/SUCCESS/FAILED/PARTIAL',
  `symbol_count` INT UNSIGNED  DEFAULT NULL
  COMMENT '处理品种数',
  `session_count` INT UNSIGNED  DEFAULT NULL
  COMMENT '生成时段数',
  `rule_matches` INT UNSIGNED  DEFAULT NULL
  COMMENT '匹配特殊日期规则数',
  `error_count` INT UNSIGNED  NOT NULL DEFAULT 0,
  `error_message` TEXT        DEFAULT NULL,
  `duration_ms` INT UNSIGNED  DEFAULT NULL,
  `triggered_by` VARCHAR(32)   NOT NULL DEFAULT 'SCHEDULER'
  COMMENT 'SCHEDULER/MANUAL/API',
  `operator`    BIGINT UNSIGNED  DEFAULT NULL,
  `created_at`  DATETIME(3)     NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
PRIMARY KEY (`id`),
UNIQUE KEY `uk_run_id` (`run_id`),
KEY `idx_target_date` (`target_date`, `status`),

```

```
KEY `idx_started_at` (`started_at`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
COMMENT=' 预计算运行日志';
```

13.2 缓存层契约



失效策略：

变更类型	失效方式	SLA
预计算完成	主动推送 Redis SET + Pub/Sub 广播 L1 失效	< 200ms
实时覆盖生效/解除	Pub/Sub 广播 L1 + L2 失效	< 100ms
规则变更	标记影响范围，下次预计算前不主动失效	次日生效

13.3 关键指标 (SLA)

指标	目标	告警阈值
下单时段判断 P99 延迟	< 1ms	> 5ms
预计算完成时间	每日 00:30 UTC 前	超时 30 分钟
覆盖生效到全网可见	< 100ms	> 500ms
t_daily_session 命中率	> 99%	< 95%
缓存命中率 (L1+L2)	> 90%	< 80%
DST 自动版本生成	每年 12 月成功	失败

十四、行业标准对齐

14.1 MIC (ISO 10383 市场识别码)

t_exchange_holiday.exchange_code 与 t_session_override.target_value (当 target_kind=EXCHANGE) 强制使用 MIC：

MIC	交易所
XNYS	New York Stock Exchange

MIC	交易所
XNAS	NASDAQ
XLON	London Stock Exchange
XJPX	Japan Exchange Group
XHKG	Hong Kong Exchanges
XSHG	Shanghai Stock Exchange
XSHE	Shenzhen Stock Exchange
XCBT	Chicago Board of Trade
XCME	Chicago Mercantile Exchange
XEUR	Eurex
BMEX	Bolsas y Mercados Españoles

完整列表见 ISO 10383 官方维护。

14.2 FIX TradingSessionStatus(35=h) 映射

本设计 `session_type` / `runtime_state` 与 FIX 协议的映射关系：

FIX TradSesStatus(340) ← `runtime_state`：

FIX 值	FIX 含义	本设计 runtime_state
1	Halted	HALT
1 + 原因=Maintenance	Halted + Maintenance	MAINTENANCE
2	Open	NULL (无覆盖)
3	Closed	NULL + state=COMPLETED
4	Pre-Open	(用 SessionSubID)
5	Pre-Close	(用 SessionSubID)
6	Request Rejected	—

FIX TradSesSubID(625) ← `session_type`：

FIX 值	FIX 含义	本设计 session_type
1	Pre-Trading	PRE_OPEN
2	Opening Auction	AUCTION (开盘竞价)
3	Continuous	OPEN
4	Closing Auction	AUCTION (收盘竞价)
5	Post-Trading	POST_CLOSE
6	Intraday Auction	AUCTION (盘中竞价)
7	Quiescent	— (用 runtime_state=HALT)

t_platform_session.fix_session_id 字段可选填，用于 FIX 网关在 TradingSessionStatusRequest 时返回对应的 TradingSessionID(336)。

14.3 CFI（ISO 10962）备注

CFI 第一字符（E=Equity, D=Debt, F=Future, O=Option, R=Rights, M=Misc, C=Collective Inv. Vehicle）建议作为品种分类树第一层的对齐参考，具体由品种模块负责。本文档仅在 path_pattern 匹配上承认这一约定。

十五、性能优化

15.1 t_daily_session 分区策略

按 trading_date RANGE 季度分区（见 8.1 DDL）。维护任务：

周期	操作
每季度初	新增下季度分区 + 滚动 p_future
每年初	把 13 个月前的分区归档到冷存（t_daily_session_archive_YYYY）
归档完成	ALTER TABLE ... DROP PARTITION p_old，秒级删除

15.2 归档与冷存储

表	在线保留	归档保留	冷存
t_daily_session	13 个月	5 年	对象存储
t_session_override	全部	—	—
t_config_audit	13 个月	7 年（MiFID II/SEC 要求）	对象存储
t_snapshot_run_log	90 天	1 年	—
t_exchange_holiday	5 年	10 年	—

冷存格式：parquet + 时间分区目录结构，便于离线分析。

15.3 缓存层架构（参考 13.2）

- **L1 本地内存**：Caffeine，按品种 + 当日，TTL 60s，命中率 > 80%
- **L2 Redis 集群**：Hash，按品种 + 当日，TTL 24h，命中率 > 95%
- **L3 MySQL**：兜底，命中即从源加载并回填 L2

15.4 写入吞吐与索引建议

表	主要写入场景	索引策略
t_daily_session	凌晨预计算批量 INSERT	主键 + uk + 2 个查询索引
t_session_override	突发写入（事件驱动）	唯一键 + 状态/时间索引
t_config_audit	任何配置变更触发	仅查询索引，不建过多索引

预计算建议批量 INSERT（1000 行/批），开启事务，避免逐行写入。

十六、完整表清单

#	表名	用途	层级
1	t_platform_session	平台每周交易时段	配置层
2	t_symbol_session	品种每周交易时段（含 DST profile）	配置层
3	t_special_session_rule	特殊日期规则	配置层
4	t_special_session	特殊日期实例	配置层
5	t_exchange_holiday	交易所假日日历（外部数据源）	配置层
6	t_dst_schedule	夏令时切换日历	配置层
7	t_daily_session	每日时段快照（按季分区）	计算层
8	t_snapshot_run_log	预计算运行日志	计算层
9	t_session_override	实时覆盖（状态切换+级联+幂等）	运行层
10	t_config_audit	配置变更审计（MiFID II / SEC）	基础设施

品种与分类相关表 (t_symbol, t_symbol_category, t_symbol_category_rel 等) 由品种模块独立维护。

十七、附录：最小化配置示例

从空库到能跑通 AAPL.US 日间交易的最小配置集：

SQL

```
-- 1. 平台周一-周五正常时段(美股 UTC 13:30-20:00, 即 EST 9:30-16:00)
INSERT INTO t_platform_session(week_day, seq_no, session_type, open_min, close_min)
VALUES (1,1,'OPEN',810,1200), (2,1,'OPEN',810,1200),
       (3,1,'OPEN',810,1200), (4,1,'OPEN',810,1200), (5,1,'OPEN',810,1200);

-- 2. AAPL.US (symbol_id=12345) 标准时段, USA DST profile
INSERT INTO t_symbol_session(symbol_id, week_day, seq_no, session_type, open_min, close_min, dst_profile)
VALUES (12345,1,1,'OPEN',870,1200,'USA'), (12345,2,1,'OPEN',870,1200,'USA'),
       (12345,3,1,'OPEN',870,1200,'USA'), (12345,4,1,'OPEN',870,1200,'USA'),
       (12345,5,1,'OPEN',870,1200,'USA');

-- 3. 2026 年 USA DST 切换日
INSERT INTO t_dst_schedule(dst_profile, year, spring_forward_date, fall_back_date, utc_offset_dst, utc_offset_std)
VALUES ('USA', 2026, '2026-03-08', '2026-11-01', -240, -300);

-- 4. 元旦休市规则
INSERT INTO t_special_session_rule(name, code, path_pattern, recur_type, month, day, is_tradable, stack_policy, priority)
VALUES ('New Year', 'NEW_YEAR', '**', 'FIXED', 1,1,0,'OVERRIDE',100);

-- 5. 感恩节当天提前 13:00 收盘 (UTC 18:00 = 1080 分钟)
INSERT INTO t_special_session_rule(name, code, path_pattern, recur_type, month, nth_week, weekday,
                                   is_tradable, open_min, close_min, stack_policy, priority)
VALUES ('Thanksgiving Early Close', 'THANKSGIVING_EARLY', 'Stocks/US/**', 'NTH_WEEKDAY', 11,4,4,
       1,870,1080,'OVERRIDE',100);

-- 6. 预计算调度任务自动生成 t_daily_session 与 t_snapshot_run_log
-- (省略, 由调度任务执行)
```

完成上述配置后：

- 元旦自动休市

- 感恩节自动提前 13:00 收盘
- DST 切换日系统自动生成新版本，运营无感
- 实时风控暂停可通过 INSERT t_session_override 立即生效

附录 B：与原设计的关键变更对照

维度	原设计	现设计
跨日时段	拆两行	单行 close_min ∈ [0,2880]
session_type 边界	HALT/MAINTENANCE 混入静态时段	静态时段类型与运行时状态类型严格分离
DST 处理	完全手动 effective_from/to 版本	dst_profile + t_dst_schedule + 自动版本生成
特殊日期 recur_type	5 种	9 种（增 LAST_WEEKDAY/ISLAMIC/JEWISH/EXPRESSION）
周末顺延	observed_forward 0/1	observed_strategy 五策略
stack_policy 默认	INTERSECT	OVERRIDE（贴近 99% 实务场景）
实时覆盖 target	target_id VARCHAR 多态	target_kind + target_value 规范化
覆盖幂等性	无	idempotency_key 唯一约束
监管披露字段	无	disclosed_at / public_announcement_id / approver
审计字段	operator	+ reason / request_id / approver / client_ip / change_source
可观测性	无	t_snapshot_run_log + 缓存契约 + SLA
行业标准对齐	隐式	显式 MIC / FIX TradSesStatus / CFI 备注
分区策略	无	t_daily_session 按 trading_date 季度分区